COURSE SLIDES

Docker Fundamentals for Beginners

Course Code: C1285

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Version 1.0

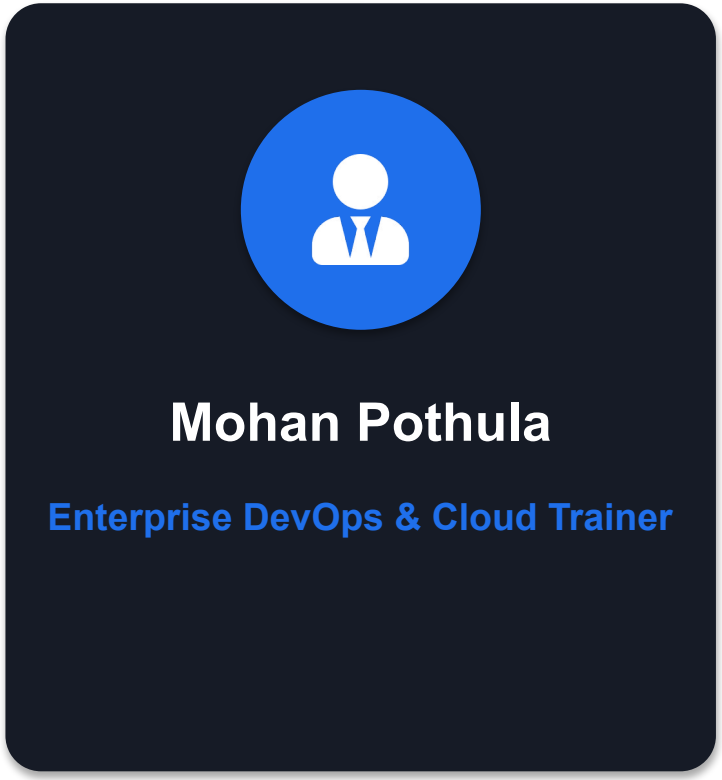
COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Role
Principal DevOps & Cloud trainer at Tertiary Infotech Academy.

Qualifications
Certified Kubernetes Administrator (CKA) · Docker Certified Associate (DCA) · AWS Solutions Architect.

Expertise
Docker · Kubernetes · CI/CD · Cloud (AWS) · platform engineering.

Experience
20+ years across DBS Bank, SingTel, Mediacorp and SPH Media.

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



[Trainer Name]

[Title / Role]



Qualifications: _____



Expertise: _____



Experience: _____



Contact / Profile: _____

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

Take a minute to introduce yourself to the class:



Your role

Your name and organisation / role.



Your experience

Any experience with Docker, servers or the cloud.



Your goal

What you want to containerise or deploy after this course.

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSE RULES

Ground Rules

Phones on silent

Set your phone to silent mode.

Participate actively

No question is too small — ask freely.

Mutual respect

One conversation at a time.

Be punctual

Return from breaks on time.

Step out quietly

For calls or short breaks.

Full attendance

Attend every session to get the most from the course.

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE

Lesson Plan — 1 Day, 7.5 hours

1 Morning

Docker

- Fundamentals & commands (Labs 1–2)
- Dockerfile, build & CMD/ENTRYPOINT (Labs 3–5)
- Storage, networking, config (Labs 6–8)
- Sharing images on Docker Hub (Lab 9)

2 Afternoon

Docker Compose & Kubernetes

- Docker Compose (Labs 10–12)
- Pods & Namespaces (Labs 13–14)
- Deployments & Rollouts (Labs 15–16)
- Services (Lab 17)
- Storage, Jobs & CronJobs (Labs 18–19)

Timing: 9:30am – 5:30pm · 30-minute lunch (12:55pm – 1:25pm)

© 2026 Tertiary Infotech Academy Pte Ltd

01

PART 1

Docker

Containers · Images · Storage · Networking · Compose

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 1

Docker Fundamentals

Containers, the Docker engine & core commands

© 2026 Tertiary Infotech Academy Pte Ltd

OVERVIEW

The Problem: 'It Works on My Machine'

- Apps break when the environment differs — OS, language version, missing dependencies or config.
- Docker packages the app AND its environment into one portable image that runs identically everywhere.
- An image is a read-only blueprint; a container is a running instance of it.
- Containers are lightweight — they share the host kernel and start in milliseconds.

© 2026 Tertiary Infotech Academy Pte Ltd

WHY CONTAINERS

Virtual Machines vs Containers

Virtual Machines

- Full guest OS per VM
- Boot time: seconds–minutes
- Size: gigabytes
- Hypervisor overhead
- Examples: VMware, VirtualBox, EC2

Containers

- Share the host OS kernel
- Start time: milliseconds
- Size: megabytes
- Minimal overhead
- Examples: Docker, containerd

© 2026 Tertiary Infotech Academy Pte Ltd

HOW IT WORKS

Docker Architecture

Docker Client

- The CLI you type
- docker run / build / ps
- Talks to the daemon

Docker Daemon

- Runs on the host
- Manages images & containers
- Networks & volumes

Registry

- Stores & shares images
- Docker Hub = default
- pull / push images

© 2026 Tertiary Infotech Academy Pte Ltd

THE CLI

Core Docker Commands

- docker run — create and start a container from an image.
- docker ps / ps -a — list running / all containers.
- docker logs / exec — read logs / run a command inside a container.
- docker stop / rm — stop / delete a container; docker images — list images.

© 2026 Tertiary Infotech Academy Pte Ltd

Docker Commands — Run & Manage Containers

LAB 1 Docker commands

Get comfortable with the core Docker commands by running a real web server (nginx) and managing its whole lifecycle: start it, list it, read its logs, run a command inside it, then stop and remove it.

You'll build

Run and manage an nginx web server with docker run / ps / logs / exec / stop / rm.

Key commands: docker run · docker logs · docker stop · » `docker

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · COMMANDS

Docker Commands — Run & Manage Containers

Part A — Run a web server in the background

```
docker run -d --name web -p 8080:80 nginx:latest
docker ps                       # the container is running
curl http://localhost:8080      # nginx serves its welcome page
```

Part B — Inspect a running container

```
docker logs web
docker exec web cat /etc/hostname
docker exec -it web /bin/bash   # open an interactive shell, then 'exit'
```

Part C — Stop, restart and clean up

```
docker stop web
docker ps -a                    # shows stopped containers too
docker start web
docker rm -f web                # force-remove (stops if running)
```


© 2026 Tertiary Infotech Academy Pte Ltd

Docker Commands — Run & Manage Containers (cont.)

Note docker ps lists only running containers; docker ps -a lists all containers including stopped ones.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · TEST IT

Docker Commands — Run & Manage Containers

✅ Test it

Open http://localhost:8080 and confirm the nginx welcome page loads while the container is running, and that docker ps no longer lists web after docker rm -f web.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

© 2026 Tertiary Infotech Academy Pte Ltd

Images & Inspecting Containers — pull, cp, inspect

LAB 2 Docker commands

Work with images and look inside containers. You will pull images, copy a file out of a running container with docker cp, and inspect container details.

You'll build

Pull images, copy a served file out of nginx with docker cp, and inspect a container.

Key commands: docker pull · docker cp · docker inspect · » `docker

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · COMMANDS

Images & Inspecting Containers — pull, cp, inspect

Part A — Work with images

```
docker pull nginx:latest
docker images                  # list local images
docker run -d --name web -p 8080:80 nginx:latest
```

Part B — Copy a file out of the container (docker cp)

```
docker cp web:/usr/share/nginx/html/index.html ./index.html
echo '<h1>Hello from TaskBoard class!</h1>' > index.html
docker cp ./index.html web:/usr/share/nginx/html/index.html
curl http://localhost:8080     # shows your new page
```

Part C — Inspect the container

```
docker inspect web | grep -i ipaddress
docker stats --no-stream web   # CPU / memory snapshot
docker rm -f web
```


© 2026 Tertiary Infotech Academy Pte Ltd

Images & Inspecting Containers — pull, cp, inspect (cont.)

Note docker cp copies files between a container and the host in either direction — handy for grabbing logs or config out of a running container.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · TEST IT

Images & Inspecting Containers — pull, cp, inspect

✅ Test it

After copying the edited file back, curl http://localhost:8080 should show your custom heading instead of the default nginx page.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 2

Dockerfile & Build

Build your own images — layers, cache, CMD vs ENTRYPOINT

© 2026 Tertiary Infotech Academy Pte Ltd

What is a Dockerfile?

- A text recipe of instructions Docker runs to build an image automatically.
- FROM (base image) · WORKDIR · COPY · RUN · EXPOSE · ENV · CMD / ENTRYPOINT.
- Each instruction creates a cached layer — unchanged layers are reused.
- Order matters: put rarely-changing steps (dependencies) before source code.

© 2026 Tertiary Infotech Academy Pte Ltd

LAYERS & CACHE

Image Layers & Build Cache

- An image is a stack of layers; each instruction adds one.
- If a layer is unchanged, Docker reuses it from cache — fast rebuilds.
- COPY requirements.txt + RUN pip install BEFORE COPY . . keeps deps cached.
- A .dockerignore keeps junk (.git, caches) out of the build context.

© 2026 Tertiary Infotech Academy Pte Ltd

INSTRUCTIONS

CMD vs ENTRYPOINT

CMD

- Default command — overridable
- CMD ["python","app.py"]
- Override: docker run img bash
- Use for a replaceable default

ENTRYPOINT

- Fixed executable — args appended
- ENTRYPOINT ["python","cli.py"]
- CMD ["list"] → default arg
- Use when the image IS a tool

© 2026 Tertiary Infotech Academy Pte Ltd

PART 1 · DOCKERFILE & BUILD

Build the TaskBoard Image with a Dockerfile

LAB 3 Docker Build / Dockerfile

Package a real application — the TaskBoard Flask web app — into your own Docker image using a Dockerfile, then run it as a container. This image is reused by several later labs.

You'll build

Write a Dockerfile for the TaskBoard Flask app, build it, and run the container.

Key commands: FROM python:3.11-slim · docker build · docker run · » Tag

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · COMMANDS

Build the TaskBoard Image with a Dockerfile

What's in the folder

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build the image

```
docker build -t taskboard:1.0 .
docker images | grep taskboard
```

Run the container

```
docker run -d --name taskboard -p 8080:5000 taskboard:1.0
curl http://localhost:8080/health
# open http://localhost:8080 and add a few tasks
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · COMMANDS

Build the TaskBoard Image with a Dockerfile (cont.)

Watch the build cache

```
docker build -t taskboard:1.0 .
docker rm -f taskboard
```

Note Tag images with a name and version (taskboard:1.0) so you can roll forward/back later — exactly what you'll do in the Kubernetes rollout lab.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · TEST IT

Build the TaskBoard Image with a Dockerfile

✅ Test it

curl http://localhost:8080/health returns {"status": "ok", ...} and the board at http://localhost:8080 lets you add a task that appears in the list.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

© 2026 Tertiary Infotech Academy Pte Ltd

PART 1 · DOCKERFILE & BUILD

Dockerfile Best Practices — .dockerignore & layer caching

LAB 4 Dockerfile best practices

Make the TaskBoard image smaller and faster to build. You'll add a .dockerignore, order instructions for maximum cache reuse, and compare image sizes.

You'll build

Add a .dockerignore and cache-friendly layer ordering to slim the TaskBoard build.

Key commands: __pycache__/ · COPY requirements.txt · docker build · » `python:3.11-slim`

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · COMMANDS

Dockerfile Best Practices — .dockerignore & layer caching

Why .dockerignore

```
__pycache__/
*.pyc
data/
.git/
.env
*.md
```

Cache-friendly ordering

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · COMMANDS

Dockerfile Best Practices — .dockerignore & layer caching (cont.)

Build and compare

```
docker build -t taskboard:slim .
docker images taskboard
# edit app.py, rebuild, and watch most layers say 'CACHED'
docker build -t taskboard:slim .
```

Note python:3.11-slim is already a smaller base than python:3.11. For even smaller images, multi-stage builds copy only the artifacts you need into a fresh final stage.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · TEST IT

Dockerfile Best Practices — .dockerignore & layer caching

✅ Test it

The second docker build completes in seconds with most steps showing CACHED, and docker images taskboard shows the image built from the slim base.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

© 2026 Tertiary Infotech Academy Pte Ltd

CMD vs ENTRYPOINT — the taskboard-cli tool

LAB 5

CMD vs ENTRYPOINT

Understand the difference between CMD and ENTRYPOINT by packaging a real command-line tool — taskboard-cli — that lists and adds tasks. ENTRYPOINT fixes the program; CMD supplies the default arguments you can override at runtime.

You'll build

Package taskboard-cli so ENTRYPOINT is fixed and CMD/args are overridable.

Key commands: FROM python:3.11-slim · docker build · docker run · » Use

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · COMMANDS

CMD vs ENTRYPOINT — the taskboard-cli tool

The Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY cli.py .
# ENTRYPOINT fixes the executable; CMD is the default argument (overridable).
ENTRYPOINT ["python", "cli.py"]
CMD ["list"]
```

Build it

```
docker build -t taskboard-cli .
```

CMD is the default; run args override it

```
docker run --rm taskboard-cli                # runs the default: list
docker run --rm taskboard-cli add "Buy milk"   # overrides CMD with: add ...
docker run --rm taskboard-cli add "Ship release"
docker run --rm taskboard-cli list
```


© 2026 Tertiary Infotech Academy Pte Ltd

CMD vs ENTRYPOINT — the taskboard-cli tool (cont.)

Note Use ENTRYPOINT when the image *is* a specific tool and arguments vary (like our CLI). Use CMD alone when you want an easily replaceable default command. Both use the JSON *exec form* ["a","b"] to avoid an extra shell.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · TEST IT

CMD vs ENTRYPOINT — the taskboard-cli tool

✅ Test it

docker run --rm taskboard-cli prints the task list, and docker run --rm taskboard-cli add "Demo" adds a task — the same image, different CMD arguments.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 3

Docker Storage

Persist data with volumes & bind mounts

© 2026 Tertiary Infotech Academy Pte Ltd

PERSISTENCE

Why Volumes?

- Data written inside a container is lost when the container is removed.
- Named volumes — Docker-managed persistent storage; best for databases & app data.
- Bind mounts — map an exact host folder in; best for live code editing in dev.
- Mount with -v name:/path (volume) or -v $(pwd)/dir:/path (bind mount).

© 2026 Tertiary Infotech Academy Pte Ltd

Docker Storage — Named Volumes & Bind Mounts

LAB 6 Volumes

Make TaskBoard's data survive container removal. You'll persist its tasks.json in a named volume, prove the data outlives the container, then use a bind mount to edit files on the host and see them live in the container.

You'll build

Persist TaskBoard tasks in a named volume; live-edit host files via a bind mount.

Key commands: docker volume · docker run · mkdir -p · » Named

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · COMMANDS

Docker Storage — Named Volumes & Bind Mounts

Part A — Named volume (persistent app data)

```
docker volume create taskboard-data
docker run -d --name tb -p 8080:5000 \
  -e DATA_DIR=/data -v taskboard-data:/data taskboard:1.0
# add a few tasks at http://localhost:8080, then:
docker rm -f tb
```

Start a brand-new container on the same volume — your tasks are still there:

```
docker run -d --name tb2 -p 8080:5000 \
  -e DATA_DIR=/data -v taskboard-data:/data taskboard:1.0
curl http://localhost:8080/api/tasks   # the tasks you added are back
docker rm -f tb2
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · COMMANDS

Docker Storage — Named Volumes & Bind Mounts (cont.)

Part B — Bind mount (live host files)

```
mkdir -p data && echo '[]' > data/tasks.json
docker run -d --name tb3 -p 8080:5000 \
  -e DATA_DIR=/data -v $(pwd)/data:/data taskboard:1.0
# add tasks in the browser, then read them straight off the host:
cat data/tasks.json
docker rm -f tb3
```

Note Named volumes are stored under Docker (/var/lib/docker/volumes) and are the right choice for databases; bind mounts tie you to an exact host path but are perfect for editing code live.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · TEST IT

Docker Storage — Named Volumes & Bind Mounts

✅ Test it

After removing tb and starting tb2 on the same named volume, curl http://localhost:8080/api/tasks returns the tasks you added earlier — they survived container deletion.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-02-docker-storage

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 4

Docker Networking

Custom bridge networks, DNS & port mapping

© 2026 Tertiary Infotech Academy Pte Ltd

Networking Concepts

- Default bridge: containers reach each other only by IP (which changes).
- Custom bridge: built-in DNS — containers reach each other by name.
- That's how a web app finds its database/cache: by container/service name.
- Port mapping (-p host:container) exposes a container to the host.

© 2026 Tertiary Infotech Academy Pte Ltd

PART 1 · DOCKER NETWORKING

Docker Networking — Custom Bridge, DNS & Port Mapping

LAB 7

Networking

Connect two containers the way real apps do: TaskBoard reaching a Redis cache by name over a custom bridge network. You'll see why a custom network (with built-in DNS) beats the default bridge, then publish the app to the host with port mapping.

You'll build

TaskBoard reaches Redis by DNS name on a custom network; publish ports to the host.

Key commands: docker network · docker run · docker exec · » Because

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · COMMANDS

Docker Networking — Custom Bridge, DNS & Port Mapping

Part A — Create a network and a Redis cache

```
docker network create tasknet
docker run -d --name redis --network tasknet redis:7-alpine
```

Part B — Run TaskBoard on the same network

```
docker run -d --name tb --network tasknet -p 8080:5000 \
  -e REDIS_HOST=redis taskboard:1.0
curl http://localhost:8080      # 'visits' counter increments on each load
curl http://localhost:8080
```

Prove the DNS name resolves from inside the app container:

```
docker exec tb getent hosts redis   # resolves to redis's IP on tasknet
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · COMMANDS

Docker Networking — Custom Bridge, DNS & Port Mapping (cont.)

Part C — Port mapping (host access)

```
docker run -d --name tb-b --network tasknet -p 8081:5000 \
  -e REDIS_HOST=redis taskboard:1.0
curl http://localhost:8081/health
docker rm -f tb tb-b redis
docker network rm tasknet
```

Note Because both TaskBoard instances share one Redis on the network, the visit counter is shared across them — your first taste of stateful, multi-container apps (which Compose and Kubernetes automate).

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · TEST IT

Docker Networking — Custom Bridge, DNS & Port Mapping

✅ Test it

Reloading http://localhost:8080 increases the visits count (served from Redis), and docker exec tb getent hosts redis resolves the redis name to an IP on the custom network.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-03-docker-networking

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 5

Configuration & Docker Hub

Runtime config & sharing images

© 2026 Tertiary Infotech Academy Pte Ltd

CONFIG & REGISTRY

Environment Variables & Registries

- Configure the same image per environment with ENV, -e and --env-file — no rebuild.
- Never bake secrets into an image; pass them at runtime.
- Docker Hub is the default public registry: docker tag → push → pull.
- **Kubernetes pulls your image from a registry by name:tag.**

© 2026 Tertiary Infotech Academy Pte Ltd

PART 1 · CONFIGURATION

Configuration with Environment Variables

LAB 8 Environment variables

Configure the SAME TaskBoard image for different environments without rebuilding. You'll set defaults with ENV in the Dockerfile, override per-container with -e, and load many values at once from an --env-file.

You'll build

Configure TaskBoard (APP_ENV, APP_TITLE) via ENV, -e and --env-file — no rebuild.

Key commands: docker run · cat .env · » Never

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · COMMANDS

Configuration with Environment Variables

1. Defaults baked into the image (ENV)

```
docker run -d --name tb -p 8080:5000 taskboard:1.0
curl http://localhost:8080/health   # "env": "development"
docker rm -f tb
```

2. Override per container with -e

```
docker run -d --name tb -p 8080:5000 \
  -e APP_ENV=staging -e 'APP_TITLE=TaskBoard (Staging)' taskboard:1.0
curl http://localhost:8080/health   # "env": "staging"
docker rm -f tb
```

3. Load many values from an --env-file

```
cat .env
docker run -d --name tb -p 8080:5000 --env-file .env taskboard:1.0
curl http://localhost:8080/health   # "env": "production"
docker rm -f tb
```


© 2026 Tertiary Infotech Academy Pte Ltd

Configuration with Environment Variables (cont.)

Note Never bake secrets into an image. Pass them at runtime with -e / --env-file (and in Kubernetes, with ConfigMaps and Secrets).

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · TEST IT

Configuration with Environment Variables

✅ Test it

The /health endpoint reports development, then staging, then production as you change only the environment variables — the image is never rebuilt.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-04-docker-config

© 2026 Tertiary Infotech Academy Pte Ltd

PART 1 · DOCKER HUB

Sharing Images — Push to Docker Hub

LAB 9

Docker Hub

Publish your TaskBoard image to Docker Hub so anyone (and any Kubernetes cluster) can pull and run it. You'll log in, tag the image under your account, push it, then pull and run it as if you were a new user.

You'll build

Tag, push and pull the TaskBoard image via Docker Hub.

Key commands: docker login · docker push · docker rmi · » This

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · COMMANDS

Sharing Images — Push to Docker Hub

Log in and tag

```
docker login
docker tag taskboard:1.0 <user>/taskboard:1.0
```

Push

```
docker push <user>/taskboard:1.0
# browse to https://hub.docker.com/r/<user>/taskboard to see it
```

Pull & run as a new user

```
docker rmi <user>/taskboard:1.0          # remove the local copy
docker run -d --name tb -p 8080:5000 <user>/taskboard:1.0
curl http://localhost:8080/health
docker rm -f tb
docker logout
```


© 2026 Tertiary Infotech Academy Pte Ltd

Sharing Images — Push to Docker Hub (cont.)

Note This is exactly how Kubernetes gets your app this afternoon: the cluster pulls your image from a registry by name:tag. Always push a versioned tag, not just latest.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · TEST IT

Sharing Images — Push to Docker Hub

✅ Test it

Your image appears at hub.docker.com under <user>/taskboard, and after removing the local image you can docker run it straight from the registry.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-01-docker-fundamentals

© 2026 Tertiary Infotech Academy Pte Ltd

Lunch Break

30 minutes

TOPIC 6

Docker Compose

Define multi-service apps in one YAML file

© 2026 Tertiary Infotech Academy Pte Ltd

MULTI-SERVICE

What is Docker Compose?

- Define all your services, networks and volumes in one docker-compose.yml.
- Compose creates a shared network; services reach each other by service name.
- Lifecycle: docker compose pull → up -d → ps / logs → down (and down -v).
- healthcheck + depends_on: condition lets the app wait for the database.

© 2026 Tertiary Infotech Academy Pte Ltd

WEB + DB + CACHE

The TaskBoard Stack

web

- TaskBoard Flask app
- Built from the Dockerfile
- Port 8080 → 5000

db (Postgres)

- Stores the tasks
- Health-checked
- pgdata volume

redis

- Visit counter / cache
- redis:7-alpine
- Reached by name

© 2026 Tertiary Infotech Academy Pte Ltd

Docker Compose — Single Service

LAB 10

Docker Compose

Replace long docker run commands with a single declarative file. You'll define TaskBoard in a docker-compose.yml and manage it with the compose lifecycle: pull, up, ps, logs, down.

You'll build

Define TaskBoard in docker-compose.yml; manage it with pull / up / ps / logs / down.

Key commands: services: · docker compose · » `docker

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · COMMANDS

Docker Compose — Single Service

The compose file

```
services:
  web:
    build: .            # build from the Dockerfile in this folder
    ports:
      - "8080:5000"     # host:container
    environment:
      - APP_ENV=development
      - APP_TITLE=TaskBoard (Compose)
```

The Compose lifecycle

```
docker compose pull        # pull any pre-built images (none to build here)
docker compose up -d       # build if needed and start in the background
docker compose ps          # service status
docker compose logs web    # logs for the web service
curl http://localhost:8080/health
```


© 2026 Tertiary Infotech Academy Pte Ltd

Docker Compose — Single Service (cont.)

Tear down

```
docker compose down        # stop and remove containers + network
docker compose down -v     # also remove named volumes (full reset)
```

Note docker compose up -d is idempotent — re-run it after editing the file and Compose only changes what's needed. One file replaces a page of docker run flags.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · TEST IT

Docker Compose — Single Service

✅ Test it

docker compose up -d starts the service and http://localhost:8080 serves TaskBoard titled 'TaskBoard (Compose)'; docker compose down removes it cleanly.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-05-docker-compose

© 2026 Tertiary Infotech Academy Pte Ltd

Docker Compose — Multi-Service (Web + Redis)

LAB 11

Docker Compose

Add a second service. Compose puts both containers on one network and gives each a DNS name equal to its service name, so TaskBoard reaches Redis at host redis with zero manual networking. The visit counter is now shared and survives restarts.

You'll build

Run TaskBoard + Redis with Compose; services find each other by name automatically.

Key commands: services: · docker compose · » `depends_on`

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · COMMANDS

Docker Compose — Multi-Service (Web + Redis)

Two services, one file

```
services:
  web:
    build: .
    ports: ["8080:5000"]
    environment:
      - REDIS_HOST=redis     # <-- the other service's name
    depends_on: [redis]
  redis:
    image: redis:7-alpine
    volumes: [redis-data:/data]
volumes:
  redis-data:
```

Run the full lifecycle

```
docker compose pull        # pulls redis:7-alpine
docker compose up -d
docker compose ps
curl http://localhost:8080 && curl http://localhost:8080  # visits go up
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · COMMANDS

Docker Compose — Multi-Service (Web + Redis) (cont.)

Prove persistence, then tear down

```
docker compose restart web
curl http://localhost:8080   # counter kept (it lives in redis-data)
docker compose down          # keep the volume
docker compose down -v       # remove the redis-data volume too
```

Note depends_on controls start order, not readiness — the next lab adds a health check so the web service waits until the database is truly ready.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · TEST IT

Docker Compose — Multi-Service (Web + Redis)

✅ Test it

Reloading http://localhost:8080 increments the visit counter, and the count survives docker compose restart web because it is stored in the redis-data volume.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-05-docker-compose

© 2026 Tertiary Infotech Academy Pte Ltd

Docker Compose — Full-Stack (Web + PostgreSQL + Redis)

LAB 12

Docker Compose

Build the complete TaskBoard stack: the web app backed by PostgreSQL for tasks and Redis for the visit counter, with a health check so the app starts only once the database is ready. This is the architecture you'll deploy to Kubernetes later today.

You'll build

Run TaskBoard + Postgres + Redis with healthchecks and condition-based depends_on.

Key commands: services: · docker compose · » This

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · COMMANDS

Docker Compose — Full-Stack (Web + PostgreSQL + Redis)

The full stack

```
services:
  web:
    build: .
    ports: ["8080:5000"]
    environment:
      - DATABASE_URL=postgresql://taskboard:secret@db:5432/taskboard
      - REDIS_HOST=redis
    depends_on:
      db: { condition: service_healthy }
      redis: { condition: service_started }
  db:
    image: postgres:16-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U taskboard"]
      interval: 5s
      retries: 5
  redis: { image: redis:7-alpine }
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · COMMANDS

Docker Compose — Full-Stack (Web + PostgreSQL + Redis) (cont.)

Run the lifecycle

```
docker compose pull        # pulls postgres + redis
docker compose up -d        # builds web, waits for db health, then starts web
docker compose ps           # db shows (healthy)
docker compose logs web
```

Verify tasks persist in Postgres

```
# add tasks at http://localhost:8080, then prove they're in the DB:
docker compose exec db psql -U taskboard -c 'SELECT id, text, done FROM tasks;'
docker compose restart web
curl http://localhost:8080/api/tasks   # tasks still there (in Postgres)
```

Tear down

```
docker compose down         # stop & remove containers + network
docker compose down -v      # also drop the pgdata volume
```


© 2026 Tertiary Infotech Academy Pte Ltd

Docker Compose — Full-Stack (Web + PostgreSQL + Redis) (cont.)

Note This web + database + cache shape is the canonical cloud app. Later today you'll deploy this very app to Kubernetes — as Pods, a Deployment, a Service, and persistent storage.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · TEST IT

Docker Compose — Full-Stack (Web + PostgreSQL + Redis)

✅ Test it

docker compose ps shows db as (healthy), tasks added in the browser appear in SELECT * FROM tasks, and they survive docker compose restart web.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day1-05-docker-compose

© 2026 Tertiary Infotech Academy Pte Ltd

02

PART 2

Kubernetes

Pods · Deployments · Services · Storage

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 7

Kubernetes Fundamentals

Architecture, the control plane & kubectl

© 2026 Tertiary Infotech Academy Pte Ltd

ORCHESTRATION

Why Kubernetes?

- Docker runs containers on one host; Kubernetes orchestrates them across a cluster.
- It self-heals (restarts failed Pods), scales, and rolls out updates with no downtime.
- Declarative: you describe desired state in YAML; Kubernetes makes it true.
- kubectl is the CLI: get / describe / apply / delete / scale / rollout.

© 2026 Tertiary Infotech Academy Pte Ltd

HOW IT WORKS

Cluster Architecture

Control Plane

- API Server
- etcd (state store)
- Scheduler
- Controller Manager

Worker Nodes

- kubelet (node agent)
- kube-proxy (networking)
- Run your Pods

Core Objects

- Pod · Deployment
- Service · Namespace
- PV / PVC · Job

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 8

Pods & Namespaces

The smallest unit & environment isolation

© 2026 Tertiary Infotech Academy Pte Ltd

FOUNDATIONS

Pods & Namespaces

- A Pod is the smallest unit — one or more containers sharing network & storage.
- Pods are ephemeral; you normally run them via a Deployment (self-healing).
- Create Pods imperatively (kubectl run) or declaratively (kubectl apply -f).
- Namespaces isolate environments (dev / staging / prod) on one cluster.

© 2026 Tertiary Infotech Academy Pte Ltd

Kubernetes Pods — Imperative & Declarative

LAB 13

Pods

Meet the smallest deployable unit in Kubernetes — the Pod. You'll create one imperatively with a single command, inspect it, then create one declaratively from a YAML file (the approach you'll use for everything afterwards).

You'll build

Create, inspect and delete a Pod both imperatively and from a pod.yaml manifest.

Key commands: kubectl run · kubectl logs · apiVersion: v1 · kubectl apply · » Pods

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · COMMANDS

Kubernetes Pods — Imperative & Declarative

Part A — Imperative (one command)

```
kubectl run web --image=nginx
kubectl get pods
kubectl get pods -o wide          # node + Pod IP
kubectl describe pod web          # events, container, volumes
```

Look inside the Pod, then remove it:

```
kubectl logs web
kubectl exec web -- cat /etc/hostname
kubectl exec -it web -- /bin/sh   # interactive shell, then 'exit'
kubectl delete pod web
```


© 2026 Tertiary Infotech Academy Pte Ltd

Kubernetes Pods — Imperative & Declarative (cont.)

Part B — Declarative (pod.yaml)

```
apiVersion: v1
kind: Pod
metadata:
  name: taskboard
  labels:
    app: taskboard
spec:
  containers:
    - name: web
      image: nginx          # swap for <user>/taskboard:1.0 to run your own image
      ports:
        - containerPort: 80
```

```
kubectl apply -f pod.yaml
kubectl get pods
kubectl describe pod taskboard
kubectl delete -f pod.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

Kubernetes Pods — Imperative & Declarative (cont.)

Note Pods are ephemeral — if a Pod dies it is not recreated. That's why you almost always run Pods through a Deployment (Lab 15), not on their own.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · TEST IT

Kubernetes Pods — Imperative & Declarative

✅ Test it

kubectl get pods shows the taskboard Pod as Running after kubectl apply, and it disappears after kubectl delete -f pod.yaml.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day2-01-k8s-pods-namespaces

© 2026 Tertiary Infotech Academy Pte Ltd

PART 2 · NAMESPACES

Kubernetes Namespaces — Environment Isolation

LAB 14

Namespaces

Use namespaces to keep environments apart inside one cluster. You'll create a dev namespace, run a Pod in it, and see how resources are isolated by namespace.

You'll build

Create a dev namespace, run a Pod inside it, and list resources per namespace.

Key commands: kubectl get · kubectl run · kubectl delete · » Namespaces

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · COMMANDS

Kubernetes Namespaces — Environment Isolation

List and create namespaces

```
kubectl get namespaces
kubectl create namespace dev
```

Run a Pod inside a namespace

```
kubectl run web --image=nginx -n dev
kubectl get pods -n dev
kubectl get pods                  # default ns: not shown here
kubectl get pods --all-namespaces
```

Clean up the whole namespace

```
kubectl delete namespace dev
```

Note Namespaces are perfect for separating dev / staging / prod (or per-team quotas) on a single shared cluster.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · TEST IT

Kubernetes Namespaces — Environment Isolation

✅ Test it

kubectl get pods -n dev lists the Pod while it exists, and kubectl delete namespace dev removes the namespace and its Pod together.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day2-01-k8s-pods-namespaces

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 9

Deployments & Rollouts

Scaling, self-healing & zero-downtime updates

© 2026 Tertiary Infotech Academy Pte Ltd

RUN AT SCALE

Deployments, ReplicaSets & Rollouts

- A Deployment manages a ReplicaSet that keeps N identical Pods running.
- Self-healing: delete a Pod and the ReplicaSet recreates it automatically.
- Scale with kubectl scale; update the image with kubectl set image.
- Rolling updates keep old + new Pods running; rollback = kubectl rollout undo.

© 2026 Tertiary Infotech Academy Pte Ltd

Deployments — Scaling & Self-Healing

LAB 15

Deployments

Run TaskBoard the production way — as a Deployment that keeps a desired number of replicas alive. You'll scale it, watch it self-heal when a Pod is deleted, and manage it both imperatively and from YAML.

You'll build

Create a Deployment, scale it, and watch Kubernetes self-heal a deleted Pod.

Key commands: kubectl create · kubectl get · apiVersion: apps/v1 · kubectl delete · » A

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · COMMANDS

Deployments — Scaling & Self-Healing

Part A — Create and scale (imperative)

```
kubectl create deployment taskboard --image=nginx
kubectl get deployments
kubectl get pods
kubectl scale deployment taskboard --replicas=3
kubectl get pods               # now 3 Pods
```

Part B — Watch it self-heal

```
kubectl get pods
kubectl delete pod <one-pod-name>
kubectl get pods               # a new Pod is already being created
```


© 2026 Tertiary Infotech Academy Pte Ltd

Deployments — Scaling & Self-Healing (cont.)

Part C — Declarative (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: taskboard
spec:
  replicas: 3
  selector:
    matchLabels:
      app: taskboard
  template:
    metadata:
      labels:
        app: taskboard
    spec:
      containers:
        - name: web
          image: nginx
          ports:
            - containerPort: 80
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · COMMANDS

Deployments — Scaling & Self-Healing (cont.)

```
kubectl delete deployment taskboard
kubectl apply -f deployment.yaml
kubectl get pods
kubectl delete -f deployment.yaml
```

Note A Deployment manages a ReplicaSet, which guarantees the desired Pod count is always running — this is the self-healing that naked Pods (Lab 13) lack.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · TEST IT

Deployments — Scaling & Self-Healing

✅ Test it

After kubectl delete pod <name>, kubectl get pods still shows 3 Pods because the Deployment recreated the missing one automatically.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day2-02-k8s-deployments

© 2026 Tertiary Infotech Academy Pte Ltd

PART 2 · ROLLOUTS & ROLLBACKS

Rolling Updates & Rollbacks

LAB 16

Rolling updates & rollbacks

Ship a new version of TaskBoard with zero downtime, then instantly roll back when something's wrong. You'll update the container image, track the rollout, and revert to a previous revision.

You'll build

Roll a Deployment forward to a new image with no downtime, then roll it back.

Key commands: kubectl create · kubectl set · kubectl rollout · » Rolling

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · COMMANDS

Rolling Updates & Rollbacks

Part A — Deploy v1, then update the image

```
kubectl create deployment taskboard --image=nginx:1.24 --replicas=3
kubectl rollout status deployment taskboard
kubectl describe deployment taskboard | grep Image
```

Update the image — Kubernetes shifts Pods gradually from the old to the new ReplicaSet (rolling update, the default strategy):

```
kubectl set image deployment/taskboard nginx=nginx:1.25
kubectl rollout status deployment taskboard
kubectl describe deployment taskboard | grep Image   # now 1.25
```

Part B — Roll back

```
kubectl rollout history deployment taskboard
kubectl rollout undo deployment taskboard            # back to 1.24
kubectl describe deployment taskboard | grep Image
kubectl rollout undo deployment taskboard --to-revision=2   # forward to 1.25 again
kubectl delete deployment taskboard
```


© 2026 Tertiary Infotech Academy Pte Ltd

Rolling Updates & Rollbacks (cont.)

Note Rolling updates keep old and new Pods running simultaneously so users never see downtime; a rollback is just a rollout to a previous revision.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · TEST IT

Rolling Updates & Rollbacks

✅ Test it

kubectl describe deployment taskboard | grep Image shows nginx:1.25 after the update and nginx:1.24 after kubectl rollout undo.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day2-03-k8s-rollouts

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 10

Services

Stable endpoints & load balancing

© 2026 Tertiary Infotech Academy Pte Ltd

NETWORKING

Why Services?

- Pod IPs change on every restart, scale or update — you can't rely on them.
- A Service gives a stable address and load-balances across matching Pods.
- ClusterIP — internal only (service-to-service).
- NodePort — opens a port on every node for external access (30000–32767).

© 2026 Tertiary Infotech Academy Pte Ltd

Services — ClusterIP & NodePort

LAB 17

Services

Give your Pods a stable address. Pod IPs change constantly; a Service provides one durable endpoint with built-in load balancing. You'll expose TaskBoard internally with ClusterIP and externally with NodePort, both imperatively and from YAML.

You'll build

Expose a Deployment with ClusterIP (internal) and NodePort (external) Services.

Key commands: kubectl create · kubectl delete · apiVersion: v1 · » ClusterIP

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 17 · COMMANDS

Services — ClusterIP & NodePort

Part A — ClusterIP (internal)

```
kubectl create deployment taskboard --image=nginx --replicas=2
kubectl expose deployment taskboard --port=80 --target-port=80
kubectl get service taskboard
kubectl get endpoints taskboard      # the Pod IPs behind the Service
```

Part B — NodePort (external)

```
kubectl delete service taskboard
kubectl expose deployment taskboard \
  --type=NodePort --port=80 --target-port=80
kubectl get service taskboard         # note the 3xxxx port
```


© 2026 Tertiary Infotech Academy Pte Ltd

Services — ClusterIP & NodePort (cont.)

Part C — Declarative (Service + Deployment YAML)

```
apiVersion: v1
kind: Service
metadata:
  name: taskboard-svc
spec:
  type: NodePort
  selector:
    app: taskboard
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

```
kubectl delete deployment taskboard service taskboard
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get svc taskboard-svc          # NodePort 30080
kubectl delete -f service.yaml -f deployment.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

Services — ClusterIP & NodePort (cont.)

Note ClusterIP is internal-only (service-to-service). NodePort opens a port on every node for outside access. In the cloud, LoadBalancer fronts NodePort with a real external IP.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 17 · TEST IT

Services — ClusterIP & NodePort

✅ Test it

kubectl get endpoints taskboard lists 2 Pod IPs behind the Service, and the NodePort Service shows a port in the 30000–32767 range.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day2-04-k8s-services

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 11

Storage, Jobs & CronJobs

Persist data & run batch work

© 2026 Tertiary Infotech Academy Pte Ltd

PERSIST & SCHEDULE

Storage & Batch Workloads

- emptyDir — scratch storage shared by containers in a Pod (lost with the Pod).
- PersistentVolume (PV) + PersistentVolumeClaim (PVC) — durable storage that outlives Pods.
- A Job runs Pods to completion (e.g. a report); it doesn't restart on success.
- A CronJob runs Jobs on a schedule (standard cron syntax).

© 2026 Tertiary Infotech Academy Pte Ltd

PART 2 · STORAGE

Kubernetes Storage — emptyDir, PV & PVC

LAB 18

Storage

Persist data in Kubernetes. You'll share a scratch directory between two containers with emptyDir, then provision durable storage with a PersistentVolume and claim it with a PersistentVolumeClaim so data survives Pod deletion.

You'll build

Share data with emptyDir; provision durable storage with a PV + PVC.

Key commands: apiVersion: v1 · kubectl apply · » This

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 18 · COMMANDS

Kubernetes Storage — emptyDir, PV & PVC

Part A — emptyDir (shared, temporary)

```
apiVersion: v1
kind: Pod
metadata:
  name: shared-pod
spec:
  containers:
    - name: writer
      image: busybox
      command: ["sh", "-c", "echo 'hello from writer' > /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
    - name: reader
      image: busybox
      command: ["sh", "-c", "sleep 5 && cat /data/message.txt && sleep 3600"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
  volumes:
    - name: shared-data
      emptyDir: {}
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 18 · COMMANDS

Kubernetes Storage — emptyDir, PV & PVC (cont.)

```
kubectl apply -f emptydir-pod.yaml
kubectl exec shared-pod -c reader -- cat /data/message.txt
kubectl delete pod shared-pod      # emptyDir data is gone
```

Part B — PersistentVolume & Claim (durable)

```
kubectl apply -f pv.yaml
kubectl apply -f pvc.yaml
kubectl get pv,pvc                 # both should show Bound
```

```
kubectl apply -f pod-with-pvc.yaml
kubectl exec pvc-pod -- cat /data/file.txt
kubectl delete pod pvc-pod
kubectl apply -f pod-with-pvc.yaml # recreate...
kubectl exec pvc-pod -- cat /data/file.txt   # ...data is still there
kubectl delete pod pvc-pod
kubectl delete -f pvc.yaml -f pv.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

Kubernetes Storage — emptyDir, PV & PVC (cont.)

Note This PV/PVC pattern is how a real TaskBoard Postgres database keeps its data in Kubernetes — storage outlives any individual Pod.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 18 · TEST IT

Kubernetes Storage — emptyDir, PV & PVC

✅ Test it

After deleting and recreating pvc-pod, kubectl exec pvc-pod -- cat /data/file.txt still prints persistent data — the PVC kept it across Pod deletion.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day2-05-k8s-storage-jobs

© 2026 Tertiary Infotech Academy Pte Ltd

Jobs & CronJobs — Batch and Scheduled Tasks

LAB 19

Jobs & CronJobs

Run work that finishes (unlike a web server that runs forever). A Job runs Pods to completion — perfect for a TaskBoard report — and a CronJob runs Jobs on a schedule, perfect for a nightly cleanup.

You'll build

Run a batch Job (report) to completion and a scheduled CronJob (cleanup).

Key commands: kubectl create · apiVersion: batch/v1 · kubectl apply · » Jobs

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 19 · COMMANDS

Jobs & CronJobs — Batch and Scheduled Tasks

Part A — Job (run to completion)

```
kubectl create job hello --image=busybox -- echo 'Hello from a Job!'
kubectl get jobs
kubectl logs job/hello
kubectl delete job hello
```


© 2026 Tertiary Infotech Academy Pte Ltd

Jobs & CronJobs — Batch and Scheduled Tasks (cont.)

Declarative — 3 completions, 2 running in parallel:

```
apiVersion: batch/v1
kind: Job
metadata:
  name: taskboard-report
spec:
  completions: 3
  parallelism: 2
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: report
          image: busybox
          command: ["sh", "-c", "echo 'Generating TaskBoard report...'; sleep 5; echo done"]
```

```
kubectl apply -f job.yaml
kubectl get pods                 # up to 2 running at once
kubectl get job taskboard-report # COMPLETIONS climbs to 3/3
kubectl delete -f job.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

Jobs & CronJobs — Batch and Scheduled Tasks (cont.)

Part B — CronJob (scheduled)

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: taskboard-cleanup
spec:
  schedule: "*/1 * * * *"     # every minute (demo); real cleanup might be nightly
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: cleanup
              image: busybox
              command: ["sh", "-c", "echo 'Cleaning up old TaskBoard sessions...'; date"]
```


© 2026 Tertiary Infotech Academy Pte Ltd

Jobs & CronJobs — Batch and Scheduled Tasks (cont.)

```
kubectl apply -f cronjob.yaml
kubectl get cronjob taskboard-cleanup
# wait ~1 minute for the first Job to be created:
kubectl get jobs
kubectl logs job/<created-job-name>
kubectl delete -f cronjob.yaml
```

Note Jobs use restartPolicy: Never and do not restart after success — unlike a Deployment, which keeps Pods running forever. CronJobs use standard cron syntax.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 19 · TEST IT

Jobs & CronJobs — Batch and Scheduled Tasks

✅ Test it

kubectl get job taskboard-report reaches 3/3 completions, and after ~1 minute the CronJob has spawned at least one Job visible in kubectl get jobs.

KillerCoda: https://killercoda.com/tertiary-labs/course/killercoda/day2-05-k8s-storage-jobs

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE CLOSING

Wrap-up & Feedback

© 2026 Tertiary Infotech Academy Pte Ltd

BEFORE YOU GO

Course Feedback



Share Your Feedback

Please complete the course feedback form before you leave — it helps us improve this course.



Certificate

Your certificate of completion will be emailed to you after the course.

© 2026 Tertiary Infotech Academy Pte Ltd

THANK YOU

Congratulations!

You took one app from a single container to a scaled Kubernetes deployment.

© 2026 Tertiary Infotech Academy Pte Ltd